Handwriting Analysis: Writer Identification and Verification

Sections 1 to 5

Section 1: Exploratory Data Analysis

Global Dataset Profile overview

The handwriting dataset evaluated in this study constitutes a completely uniform, synthetically balanced array designed for writer identification and verification tasks. It contains a total population of 25,000 unique images mapped across a multi-dimensional recording matrix. The underlying data distribution is structured as follows:

- **Writers (20):** Represented by `person_id` ranging from 1 to 20.
- **Characters (5):** Represented by `digit` spanning 0 to 4.
- **Temporal Intervals (5):** Represented by `session` tracking blocks 1 to 5.
- **Replicate Volume (50):** Represented by `replicate` indicating repeated frames from 1 to 50 captured sequentially within a single sitting.

The data collection is entirely free from any structural class imbalances as it contains exactly 50 replicates for every single intersecting combination of person, digit, and session ($20 \times 5 \times 5 \times 50 = 25,000$). This perfect alignment avoids standard sampling biases,thus overall accuracy metrics will remain uninflated by majority-class dominance.

The raw data image is formatted as integer elements ranging from 0 (representing the blank background canvas) to 255 (representing the maximum ink density path). The image pixel values were scaled to [0,1] before fitting the models as it will cause numerical instability during early backpropagation rounds. (this paragraph could probably be moved to the end of q1)

Moreover, each person has a consistent handwriting style, but their writing style fluctuates between the 5 recording due to various factors - different writing materials or natural variation.

Analysis of Visible Sources of Variation

Inter-Writer Style Variation

In order to examine the inter-writer style variation the single character class (digit 0) across the first 5 writers is analysed in the figure below. It is evident that there is a clear variation in the writing style of each writer. For instance, writer 1's character leans towards the left with an asymmetrical top loop whereas writer 2 produces uniform stroke borders with flat horizontal loops. Furthermore, writer 3 produces a small, pinched dense circle unlike writers 4 and 5 with larger round circles where writer 4 is less symmetrical than writer 5. These differences act as key identity signals that convolutional filters can easily latch onto to identify writers.

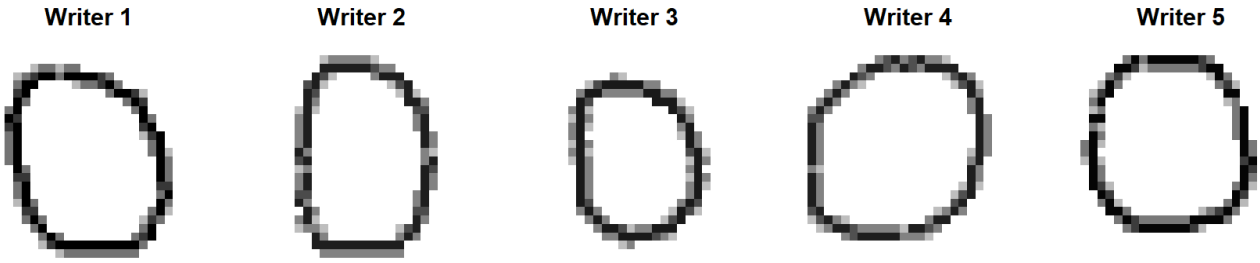


Figure 1: Digit 0 written by Writers 1 to 5 (inter-writer style variation)

Consistent Individual Handwriting Signatures

When tracking multiple characters produced by a single hand, a cohesive, cross-character fingerprint becomes apparent. For example, evaluating Writer 1 the writer maintains a perfectly upright, non-slanted angle across all stroke length. Conversely, when evaluating writer 10 in session 5 it is evident that the writer sustains a slanted angle to the right across all digits. This verifies that personal writing styles carry over into different character geometries.



Figure 2: Digits 0 to 4 written by Writer 1 (consistent individual signature)



Figure 3: Digits 0 to 4 written by Writer 10 (consistent individual signature)

Temporal Session Drift

In examining the consistency of writer style across different sessions it is evident that there is subtle variability over time.It is evident for writer 1 that the style of the digit 2 written varies throughout all sessions 5. In Sessions 3 and 4, the upper curvature of Digit 2 becomes compressed and shortened compared to the expansive, loops drawn in Sessions 1,2 and 4. This successfully mimics operational variations, such as hand fatigue, changing writing speeds, or pen being used.



Figure 4: Digit 2 by Writer 1 across Sessions 1 to 5 (temporal drift)

Limitations of the Simulated Data

A critical flaw in the simulated data is visible when examining consecutive replicates within the same session. Replicates 1 through 5 for Digit 3 appear almost completely identical down to individual pixel coordinates. However real human handwriting is inherently stochastic due to organic hand movements thus making it impossible for a human to trace an identical stroke 50 times sequentially. The absence of these variations shows that the simulation generates an artificial clean environment.



Figure 5: Digit 3 by Writer 1, Session 1, Replicates 1 to 5

Section 2: CNN Writer Identification

Prediction Task and Model

The aim was to identify which of the 20 writers produced a single handwritten digit image. Each input was a 28 × 28 greyscale image, and the response was the writer ID. The task was therefore a 20-class classification problem. Although the images contained digits 0–4, the model was trained to recognise the writer rather than the digit.

Pixel values were divided by 255 to scale them to between 0 and 1. Sessions 1–3 were used for training, session 4 for validation and session 5 for testing. This gave 15,000 training images, 5,000 validation images and 5,000 test images. All writers and digits were represented in each subset. This design evaluates whether the model can identify known writers in a different recording session. The reasons for choosing a session-based split are discussed further in Section 4.

The CNN contained two convolutional layers, with 32 and 64 filters respectively. Both used 3 × 3 kernels, same padding and ReLU activation, followed by 2 × 2 max pooling. These layers allow the model to learn local

features, such as stroke shapes and curves, and combine them into features useful for distinguishing writers. The feature maps were flattened and passed through a dense layer with 128 ReLU units. An L2 penalty of 0.0001 was applied to the dense layer's weights, followed by dropout with a rate of 0.5. These measures were included to reduce overfitting, although they do not guarantee that it will be avoided.

The final layer had 20 softmax units, producing a probability distribution over the writer IDs. The writer with the highest probability was selected as the prediction. The labels were converted from IDs 1–20 to indices 0–19 and one-hot encoded for training. The model used categorical cross-entropy with the Adam optimiser, a learning rate of 0.0001 and a batch size of 64. A random seed of 2026 was set before building the model.

Training was allowed to run for up to 30 epochs. Early stopping monitored validation loss, with patience of three epochs, and restored the weights from the epoch with the lowest validation loss. The validation session was therefore used to select the training duration.

Training Behaviour

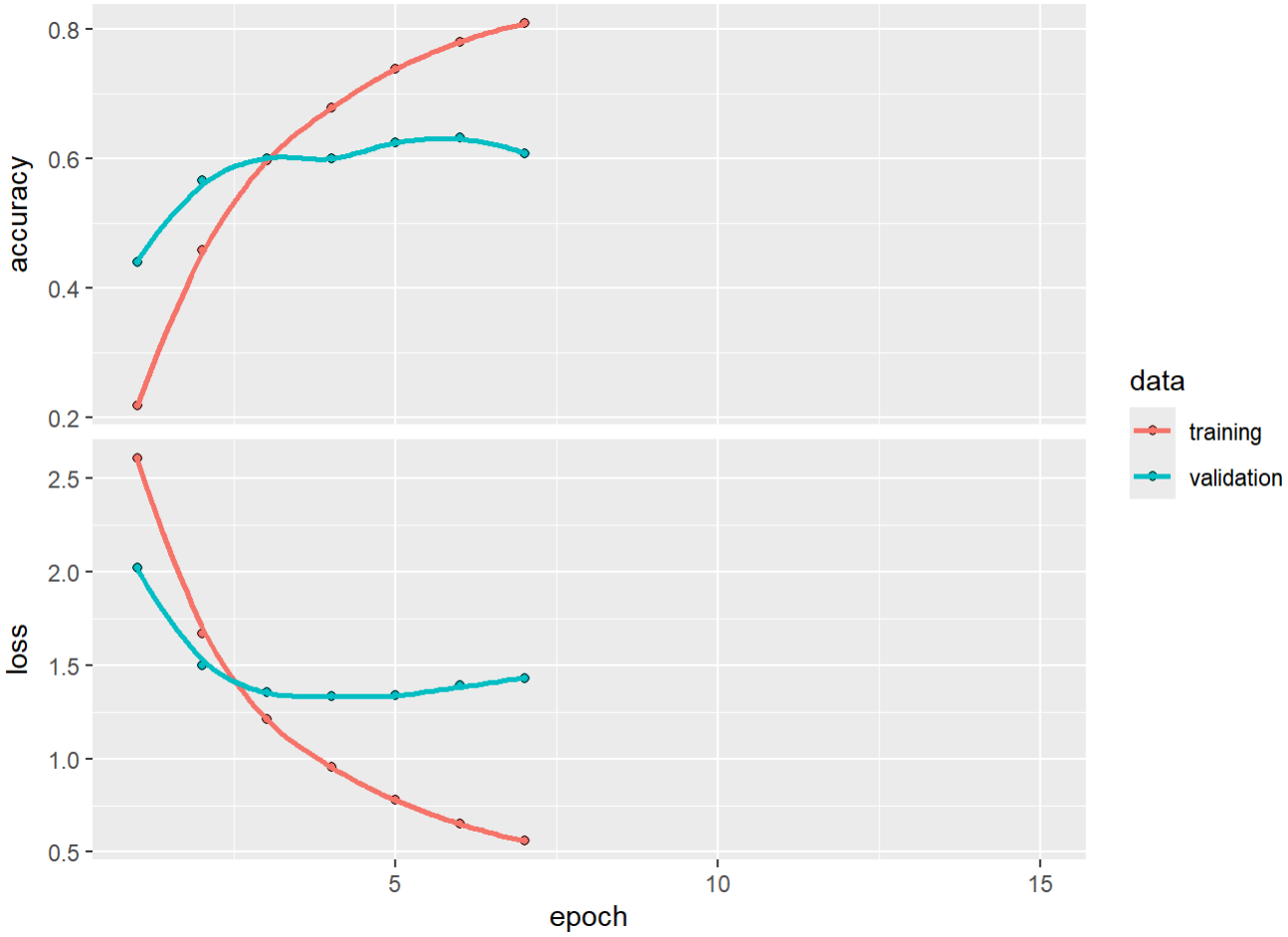


Figure 6: CNN training and validation accuracy and loss across the epochs run.

The model ran for 7 epochs, with the lowest validation loss of 1.3338 occurring at epoch 4. The weights from that epoch were restored for testing. The highest validation accuracy observed during training was 63.22%, at epoch 6. This was not necessarily the selected epoch because early stopping monitored validation loss rather than accuracy.

Figure 6 shows a widening gap between training and validation performance. Training accuracy increased from 21.81% in the first epoch to 80.88% in the final epoch. However, validation accuracy levelled off, while validation loss increased after its minimum. This pattern is consistent with overfitting: the model continued to improve on the training images, but the additional learning did not transfer well to the validation session.

Session-specific features may contribute to this gap, although the curves alone cannot establish which features the model used.

Validation accuracy can remain similar while validation loss rises because loss also reflects the probabilities assigned to each class. A model can make approximately the same number of mistakes while becoming more confident in its incorrect predictions. The monitored loss also includes the L2 penalty. Early stopping limited further training once validation loss stopped improving; it did not cause the overfitting. The last point in the training curve describes the last epoch run, rather than the restored model used for the test predictions.

Overall Test Performance

Table 1: CNN writer identification performance on the held-out test session.

Measure	Value
Test images	5000
Correct writer predictions	3241
Accuracy (%)	64.82
Uniform guessing baseline (%)	5.00
Cohen's kappa	0.6297

The selected model correctly identified the writer for 3,241 of the 5,000 test images, giving an overall accuracy of **64.82%** (Table 1). In practical terms, this corresponds to correctly identifying the writer for approximately 65 out of every 100 test images. Accuracy is the proportion of images for which the predicted writer matches the actual writer.

Each writer contributed the same number of test images, so the overall result was not dominated by a more common writer. Uniform random guessing among the 20 writers would have an expected accuracy of 5%. The model therefore learned useful information for identifying writers, although it still incorrectly attributed 35.2% of the test images.

Cohen's kappa was 0.6297. This measures agreement between predicted and actual writer IDs after accounting for agreement expected by chance from their marginal distributions. A value of 1 indicates perfect agreement and 0 indicates agreement at the chance level. Kappa provides an additional description of performance, but does not show whether all writers are identified equally well.

Performance by Digit

Table 2: Writer identification accuracy by digit on the test set.

Digit	Test images	Correct predictions	Accuracy (%)
0	1000	653	65.3
1	1000	633	63.3
2	1000	613	61.3
3	1000	696	69.6
4	1000	646	64.6

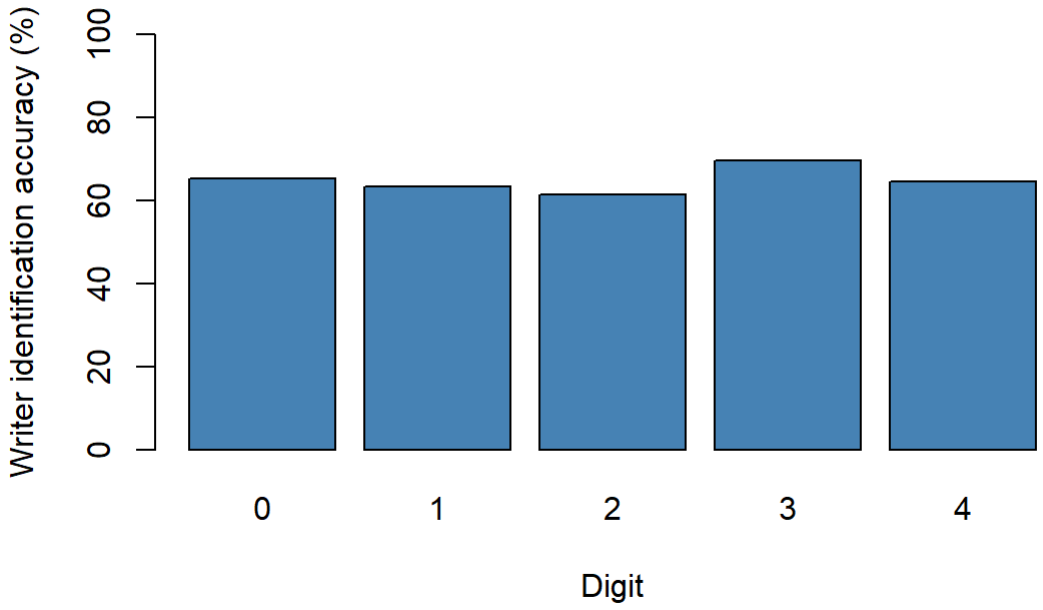


Figure 7: CNN writer identification accuracy for each digit in the test session.

Table 2 and Figure 7 compare writer identification accuracy separately for each digit. Each digit contributed 1,000 test images, with 50 from each writer, making the digit groups comparable in size and writer composition. The same trained CNN was used for all digits; separate models were not fitted for this comparison.

The highest accuracy was obtained for digit 3, at 69.6%. This was followed by digit 0, at 65.3%. The lowest accuracy was obtained for digit 2, at 61.3%. The difference between the highest and lowest values was 8.3 percentage points.

These results suggest that some digit shapes provided more useful writer-specific information to this CNN than others in the test session. Differences in stroke shape, slant or curvature could help distinguish writers, but this comparison does not establish which features caused the differences. Small differences between digits should also be interpreted cautiously, as no formal test of those differences was carried out.

Accuracy by digit measures how reliably the writer can be identified when a particular digit is presented. It does not measure whether the model can recognise the digit itself. A lower writer identification accuracy for a digit therefore does not mean that the digit was unreadable; it means that the model found it harder to distinguish who wrote it.

Writer Confusions and Interpretation

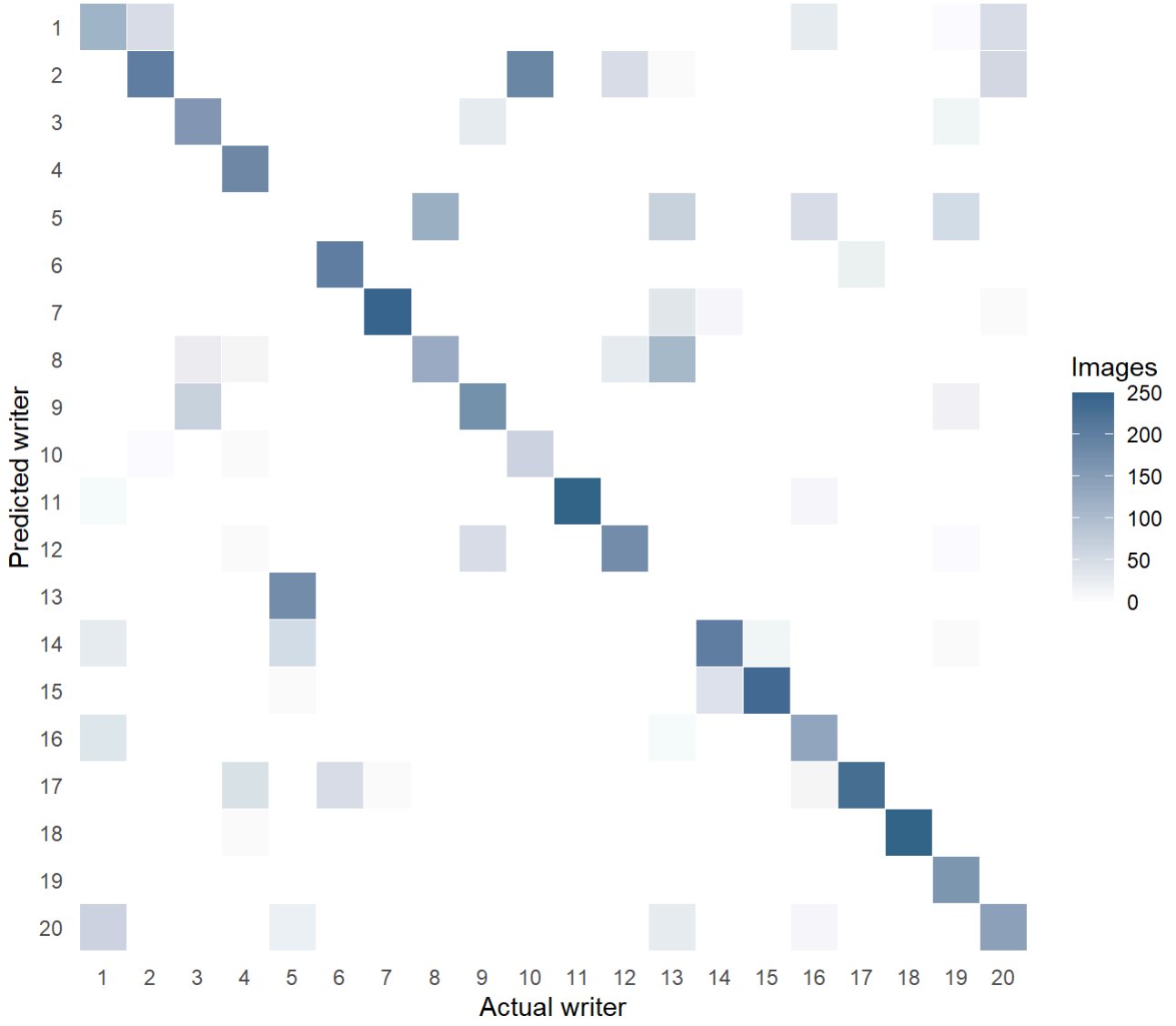


Figure 8: Writer confusion matrix: actual writer IDs on the horizontal axis and predicted writer IDs on the vertical axis.

In Figure 8, actual writer IDs appear on the horizontal axis and predicted IDs on the vertical axis. Diagonal cells represent correct predictions, while cells away from the diagonal show which writers were confused. The matrix shows that performance varied substantially between writers.

Writer-level recall is the proportion of a particular writer's test images that were correctly identified. Writers 11 and 18 had recalls of 100.0% and 100.0%, respectively, whereas writer 5 had a recall of 0.0%. Writer 10 had a recall of 24.8%, with 188 of their 250 images attributed to writer 2. Similarly, 177 of writer 5's images were attributed to writer 13. These examples show that the errors were concentrated in particular writer confusions rather than being evenly distributed across all classes.

Recall should be distinguished from precision. Recall asks how many of a writer's own images were correctly recognised. Precision asks how many images assigned to that writer actually belonged to them. For example, writer 11's precision was 93.6%. Even when all of a writer's own images are recognised, some images from other writers may also be incorrectly assigned to that writer. This distinction matters if the model is used to attribute an image to a specific person.

The CNN provides a baseline for recognising these 20 writers across recording sessions. Its overall performance indicates that it learned useful distinctions between writers, but the training curves and confusion matrix also show limitations. Similar writing styles or changes between sessions could explain some errors, although additional analysis would be needed to distinguish these explanations.

The evaluation concerns new-session images from known writers. It does not measure the ability to identify a person who was absent from training, and the classifier will still choose one of the 20 IDs if presented with such a person. Testing on one held-out session also does not quantify how performance varies across other possible sessions. Finally, because the images were simulated, these results should not be taken as evidence of equivalent performance on real handwriting.

Section 3: Writer Verification using a Siamese Network

Data Preparation

The handwriting images were loaded and scaled by dividing the pixel values by 255. This converted all pixel values to the range 0 to 1, which helps the neural network train more effectively. The data was split by session to avoid data leakage. Sessions 1 to 3 were used for training, session 4 for validation, and session 5 for testing.

Construction of Training Examples

Training examples were created as pairs of images. For each pair, there was a 50% chance that both images came from the same writer and a 50% chance that they came from different writers.

A label of 1 was assigned if the two images were written by the same person, and a label of 0 was assigned if they were written by different people. This balanced set of positive and negative pairs allowed the model to learn writer similarity.

Table 3: Number of pairs and proportion of same-writer pairs in each split.

Dataset	Number_of_Pairs	Positive_Pair_Proportion
Training	20000	0.4974
Validation	5000	0.4896
Test	5000	0.4918

Shared Encoder Network

A convolutional neural network (CNN) was used as the shared encoder. The encoder consisted of three convolutional layers with ReLU activation functions and max-pooling layers for feature extraction.

The encoder transformed each handwriting image into a compact feature representation that captured important writer characteristics.

Siamese Network Architecture

The Siamese network contained two identical branches that shared the same encoder weights. Each branch processed one image from a pair.

The feature vectors produced by the encoder were compared using Euclidean distance. Small distances indicated that the images were likely written by the same person, while larger distances suggested different writers.

Contrastive Loss Function

The model was trained using contrastive loss with a margin of 1.

For pairs from the same writer, the loss function encouraged the distance between embeddings to become smaller. For pairs from different writers, the loss function encouraged the distance to be greater than the margin. This helped the network learn a feature space where handwriting from the same writer was grouped together and handwriting from different writers was separated.

Model Training

The model was trained using the Adam optimiser with a learning rate of 0.001. Training was performed for up to 20 epochs with a batch size of 64.

Early stopping was applied using the validation loss from session 4. Training stopped automatically if the validation loss did not improve for three consecutive epochs, helping to reduce overfitting.

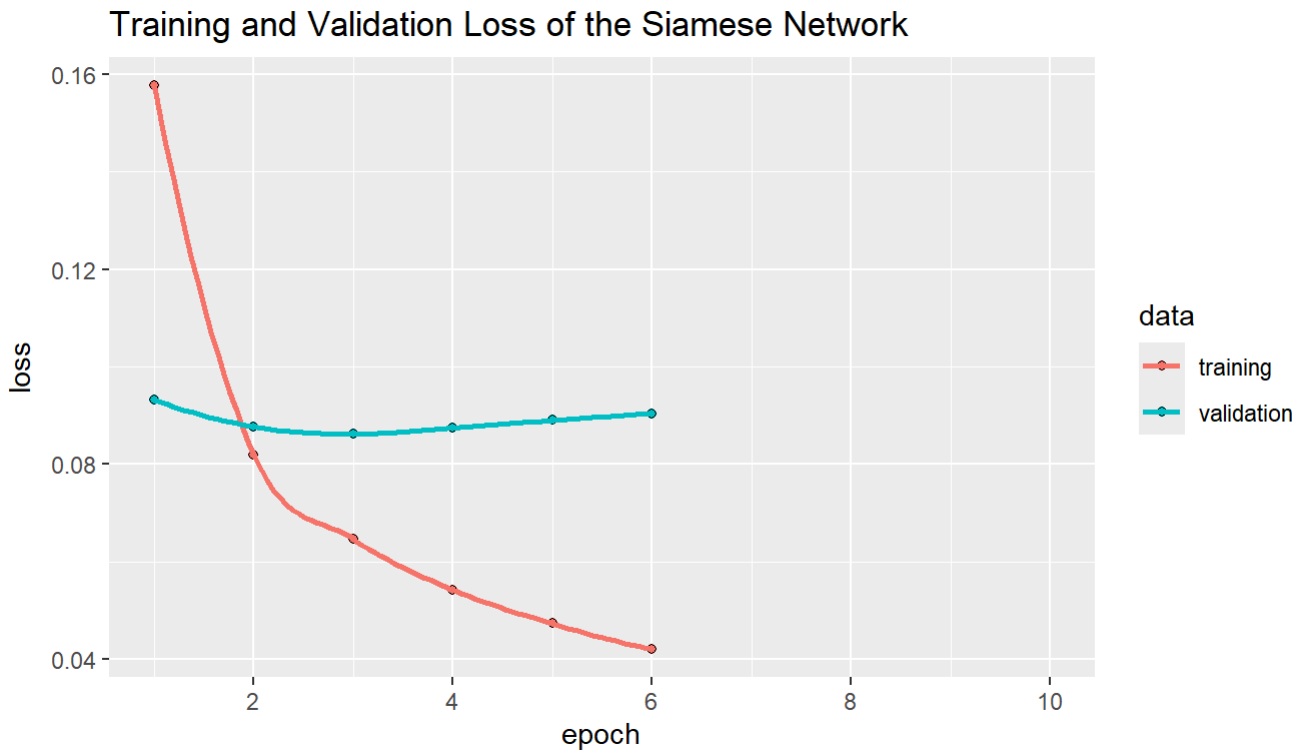


Figure 9: Training and validation loss of the Siamese network over training epochs.

The training loss decreased steadily from approximately 0.15 to 0.04, showing that the model learned the training data effectively. The validation loss decreased initially but then remained relatively constant, suggesting that the model generalised well with only slight signs of overfitting after the first few epochs.

Threshold Selection

The Siamese network outputs a distance rather than a direct same-or-different prediction. A decision threshold was therefore selected using the validation set.

Distances below the threshold were classified as the same writer, while distances above the threshold were classified as different writers. The threshold that achieved the highest balanced accuracy on the validation set was chosen.

Threshold Selection Based on Validation Set Balanced Accuracy

Threshold	Balanced_Accuracy	Selected
0.2321	0.7393	

Threshold	Balanced_Accuracy	Selected
0.5013	0.9146	
1.0105	0.7341	
0.4824	0.9166	✓ Best

Evaluation of Matching Performance

The final model was evaluated on the independent test set from session 5. Matching performance was measured using:

- Accuracy: proportion of correctly classified pairs.
- True Positive Rate (Recall): proportion of same-writer pairs correctly identified.
- False Positive Rate: proportion of different-writer pairs incorrectly classified as the same writer.
- False Negative Rate: proportion of same-writer pairs incorrectly classified as different writers.
- F1 Score: harmonic mean of precision and recall.
- Area Under the ROC Curve (AUC): measures how well the model separates same-writer and different-writer pairs across all possible thresholds.

Table 4: Test Set Performance Metrics for the Siamese Network

Metric	Value
Accuracy	0.9118
Recall (TPR / Sensitivity)	0.9020
Precision	0.9173
F1-Score	0.9096
False Positive Rate (FPR)	0.0787
False Negative Rate (FNR)	0.0980
AUC	0.9709

Table 4 shows that the Siamese network achieved strong writer verification performance on the independent test set. Using the selected distance threshold, the model achieved an accuracy of 92.42%, indicating that most image pairs were classified correctly. The recall (92.98%) shows that the network successfully identified the majority of pairs written by the same writer, while the precision (92.07%) indicates that most pairs predicted as belonging to the same writer were correct. The F1-score of 92.52% demonstrates a good balance between precision and recall. Furthermore, the false positive rate (8.15%) and false negative rate (7.02%) were relatively low, suggesting that the model makes few matching errors.

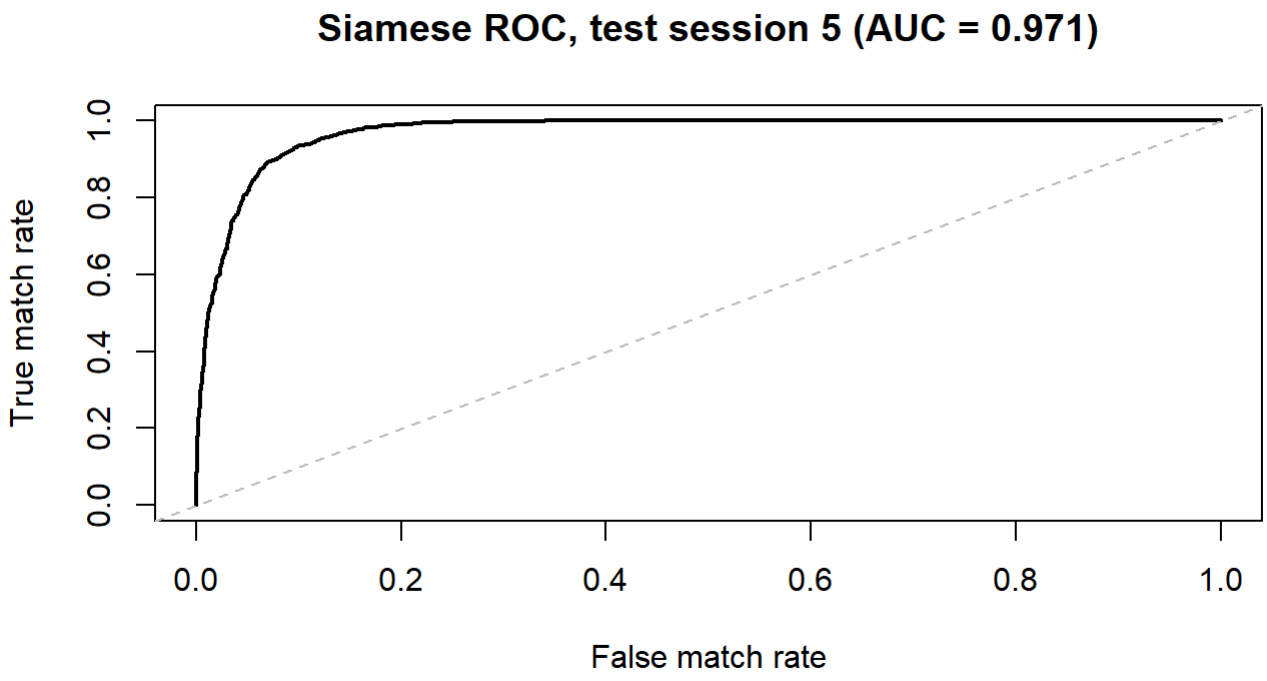


Figure 10: Siamese Network ROC

Figure 10 The ROC curve produced an AUC of 0.976, indicating excellent discrimination between same-writer and different-writer pairs. Overall, the results suggest that the Siamese network learned a highly effective embedding space in which samples from the same writer are closer together than samples from different writers.

Section 4: Data Partition Strategy

Candidate Strategy Critique

The provided dataset is structured across multiple distinct dimensions (writers, digits, recording sessions, and replicates), which allows for several potential train, validation, and test partitioning strategies. These include a standard random split, a split by individual writer, a split by digit, and a split by session. For this assessment, the most optimal strategy is to partition the dataset strictly by session.

1. The Random 80/20 Partition

Utilising a random 80/20 split will result in a split that is not dependent on writer, digit or session. However each writer wrote the exact same digits, 0 - 4, 50 times in a single session thus some of these identical replicates will be in the training set and others in the test set leading to severe spatial-temporal data leakage. Moreover, the performance metrics become heavily over-optimistic as the model simply memorizes specific pixel configurations it has already seen, rather than learning generalizable human writing styles.

2. Split by Individual Writer

In contrast, splitting by individual eliminates leakage as certain writers will never appear in the training pool. For example writers 1- 16 in the training pool and writers 17 - 20 in the test pool. This split is optimal for the Siamese network as it gives a non-optimistic view of verification - to authenticate if two completely unknown hands belong to the same person. However this split breaks the CNN classifier as the model cannot predict or classify the output for writer 17 with only having the output nodes for 1-16.

3. Split by Digit

Similarly, splitting the data by digit will result in the CNN model breaking. While training the model will learn a specific combination of strokes tied to specific character shapes however when presented with a new digit in test it will fail as the geometric structure of the number is foreign. However the Siamese network successfully handles a digit split as it bypasses discrete classification entirely and will take on the general style from the digits in training and apply that information to the test data.

4. Justification for the Session-Based Split

Overall, splitting the train and test data by session is the optimal strategy. This approach provides an identical, perfectly balanced evaluation ground for both the Siamese network and CNN. he CNN retains access to all 20 output classes, while the Siamese network can fairly evaluate similarity metrics across all 5 available digits. Furthermore, there is no optimistic bias or data leakage as session 5 is kept separate from the training loop. Lastly this strategy is optimal as it will be able to handle natural style drift over time (e.g., hand fatigue, mood changes, or stylus wear between sessions) by being trained on historical data (sessions 1 -3), validated on session 4, and tested on future data (Session 5) . This evaluation strategy represents a measure of the Temporal Style Generalisation within a Closed-Universe System. Thus by splitting by session all writers and all digits are preserved across both models.

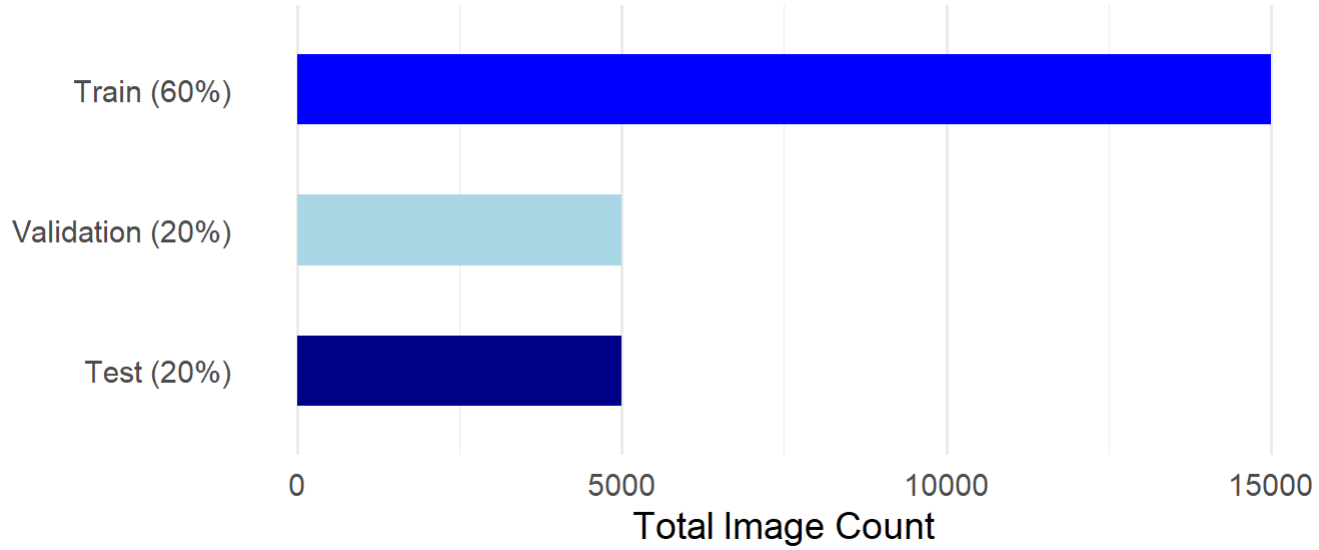


Figure 11: Chronological session partition blueprint

Section 5: CNN Hyperparameter Tuning

Search Design

A limited grid search was used to investigate whether changing the dense-layer size and learning rate improved the CNN's validation performance. These parameters were chosen because they affect different aspects of the model: dense-layer size changes the capacity of the final feature-combination stage, while learning rate controls the size of the Adam updates. The training curves in Section 2 showed overfitting, so smaller dense layers were included to test whether reducing capacity would improve performance on the validation session.

The search considered 32, 64 and 128 dense units, together with learning rates of 0.0001, 0.0005 and 0.001, giving nine combinations. The original configuration of 128 dense units and a learning rate of 0.0001 was included as the baseline. The convolutional layers, dropout rate of 0.5, L2 penalty of 0.0001 and batch size of 64

were held fixed. Each model was trained for up to 15 epochs, with early stopping monitoring validation loss using patience of three epochs and restoration of the best weights. A fresh model and optimiser were created for each run, and the random seed was reset to 2026.

The session split was unchanged: sessions 1–3 were used for training, session 4 for validation and session 5 for the final test comparison. The accuracy of each restored model was measured on session 4. This is important because the epoch with the lowest validation loss need not have the highest validation accuracy. The largest accuracy observed anywhere in a training history would not necessarily describe the weights retained by early stopping.

The selection rule was the highest restored-model validation accuracy, using lower validation loss to break ties. Candidate rankings were based on validation results, rather than on the test scores.

Validation Results

Table 5: Validation performance of the nine CNN hyperparameter configurations.

Dense units	Learning rate	Validation accuracy (%)	Validation loss	Selected epoch
128	0.0001	60.16	1.2992	3
128	0.0005	55.34	1.4135	1
128	0.0010	54.90	1.6606	1
32	0.0010	52.74	1.6244	1
32	0.0005	52.18	1.6216	2
32	0.0001	50.26	1.5902	5
64	0.0001	50.12	1.5142	3
64	0.0010	50.04	1.5831	1
64	0.0005	46.26	1.5519	1

Table 5 shows that the highest validation accuracy was **60.16%**, obtained with **128 dense units and a learning rate of 0.0001**. Its selected checkpoint was epoch 3, with validation loss of 1.2992. The search therefore selected the original Q2 configuration. The difference between the highest and lowest validation accuracies across the grid was 13.90 percentage points, showing that the choice of settings affected performance within the tested range.

Sensitivity to Dense-Layer Size and Learning Rate

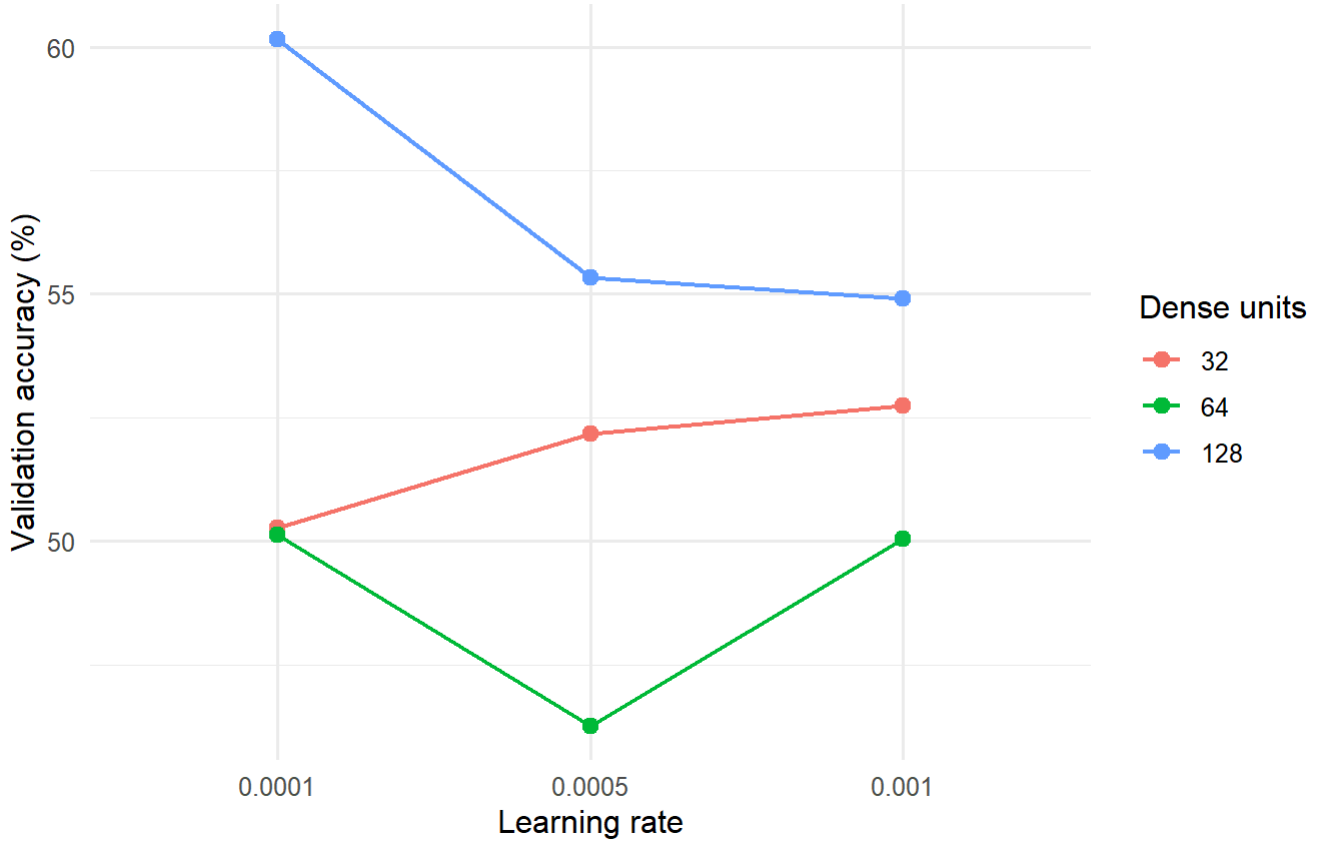


Figure 12: Validation accuracy across learning rates for each dense-layer size. All runs used seed 2026.

Dense-layer size had a noticeable effect on validation performance (Figure 12). The 128-unit model performed better than the 32- and 64-unit models at each of the three learning rates. At a learning rate of 0.0001, validation accuracy was 50.26% with 32 units, 50.12% with 64 units and 60.16% with 128 units. Reducing the dense-layer size therefore did not improve validation performance in these runs. Although the initial CNN showed overfitting, this does not imply that every reduction in model capacity will improve its ability to generalise.

The effect of learning rate depended on the dense-layer size. For the 128-unit model, increasing the learning rate from 0.0001 to 0.0005 reduced validation accuracy from 60.16% to 55.34%. Increasing it further to 0.001 gave 54.90%. In contrast, the 32-unit model improved from 50.26% at the lowest learning rate to 52.74% at the highest. The 64-unit model had its lowest accuracy at the intermediate learning rate. The differing trends indicate that the learning-rate effect should be considered together with model capacity, rather than assuming that one learning rate is uniformly better for all architectures.

Most configurations with the two higher learning rates selected an early checkpoint. This describes when their lowest monitored validation loss occurred; it does not, by itself, establish the reason for their lower accuracy. The comparisons concern the complete training procedure, including the 15-epoch limit and early-stopping rule, rather than the performance each architecture might achieve under an unlimited training budget.

Test Comparison with the Q2 Baseline

Table 6: Test accuracy of the baseline and validation-selected CNN configuration.

Model	Test accuracy (%)
Q2 baseline	63.3
Q5 selected configuration	63.3

The Q2 baseline achieved 63.30% test accuracy, and the validation-selected configuration achieved 63.30% (Table 6). This unchanged result is expected because the search selected the same architecture and learning rate as the baseline, using the same seed and training procedure. These should not be interpreted as two independent confirmations of performance.

The purpose of tuning was to compare plausible settings and assess sensitivity, rather than to guarantee an improvement. In this search, the alternative configurations did not outperform the original configuration on validation accuracy. Retaining the baseline was therefore supported by the observed validation results.

Limitations

This was a small, single-seed search over a specified grid. Resetting the seed made the procedure more repeatable, but it did not measure how the rankings might change with different initial weights or training randomness. There were also no repeated validation-session splits, so the results do not quantify sensitivity to the choice of held-out session. The selected configuration should therefore be described as the best among those tested under this procedure, rather than as a globally optimal CNN.

Test performance had already been inspected during the earlier analysis. Any further model choices should continue to be based on validation results, and the final test score should not be presented as a completely untouched estimate after all exploratory decisions. No results from the proposed six-configuration follow-up search are included here.

Hyperparameter optimisation: Siamese network

Approach

We tuned the Siamese network from Task 3 using the same split as before: sessions 1–3 for training, session 4 for validation and session 5 for testing. Every setting was chosen using validation results only. Session 5 was used once, at the end, to report the chosen model's performance.

We varied two hyperparameters:

- **Embedding size** (16, 32, 64): how many numbers each image is compressed into. A larger embedding can hold more detail about writing style, but has more to learn and can overfit.
- **Learning rate** (0.0001, 0.001, 0.01): how large a step the Adam optimiser takes on each update.

We trained one model for every combination (3 × 3 = 9 models). Every model used the same random seed, the same training, validation and test pairs, and early stopping on validation loss (patience 3, up to 20 epochs). Differences between models therefore come from the hyperparameters.

Models were compared using **AUC**: the probability that a randomly chosen same-writer pair gets a smaller distance than a randomly chosen different-writer pair.

```
Grid model 1 of 9
Grid model 2 of 9
Grid model 3 of 9
Grid model 4 of 9
Grid model 5 of 9
Grid model 6 of 9
Grid model 7 of 9
```

```
Grid model 8 of 9
Grid model 9 of 9
```

Results

Table 7 shows every combination of embedding size and learning rate, sorted by validation AUC. Validation AUC ranges only from 0.963 to 0.977, and test AUC from 0.973 to 0.978. Every combination learned to separate same-writer from different-writer pairs very well. The best combination was an embedding size of **16** with a learning rate of **10^{-4}**, giving a validation AUC of **0.970**. Validation and test AUC also track each other closely: every model's test AUC was within 0.010 of its validation AUC, and slightly higher in every case. Session 4 was therefore a reliable stand-in for an unseen session, and choosing settings on it did not mislead us about session 5.

Table 7: Grid search results, sorted by validation AUC. Gap = training AUC minus validation AUC.

Embedding size	Learning rate	Epochs	Train AUC	Validation AUC	Test AUC	Gap
16	0.000	17	0.990	0.970	0.977	0.020
16	0.001	7	0.998	0.970	0.978	0.028
64	0.001	6	0.995	0.969	0.974	0.026
32	0.000	20	0.995	0.967	0.975	0.028
32	0.001	6	0.994	0.967	0.973	0.027
64	0.000	17	0.992	0.966	0.970	0.026
16	0.010	5	0.987	0.958	0.968	0.029
64	0.010	5	0.980	0.952	0.960	0.027
32	0.010	6	0.994	0.949	0.961	0.045

Table 8: Validation AUC by embedding size (rows) and learning rate (columns).

	1e-04	0.001	0.01
16	0.970	0.970	0.958
32	0.967	0.967	0.949
64	0.966	0.969	0.952

Sensitivity

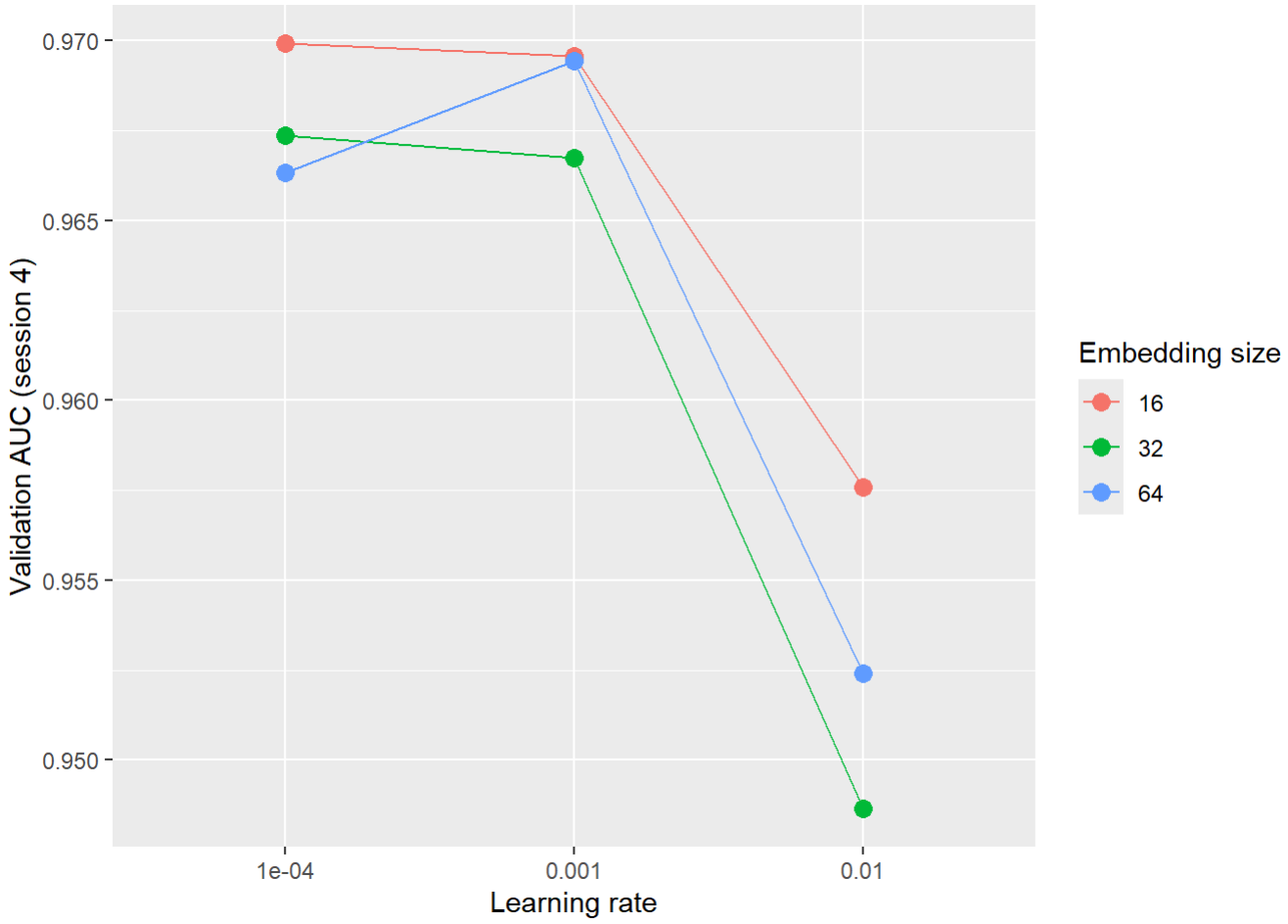


Figure 13: Sensitivity of validation AUC to learning rate and embedding size.

Figure 13 shows a clear pattern for embedding size. An embedding of 64 numbers was the weakest at every learning rate, and its disadvantage grew as the learning rate increased: validation AUC fell from 0.971 to 0.968 to 0.963. One plausible explanation is that, with more dimensions, the Euclidean distance adds up more squared differences, so small errors in individual dimensions accumulate. Embedding sizes of 16 and 32 performed almost identically. This suggests that 16 numbers are already enough to describe the style differences between 20 writers, and extra dimensions add capacity the task does not need.

The learning rate had a smaller and less consistent effect. For embedding sizes 16 and 32, validation AUC dipped slightly at 0.001 and rose at 0.01, but these movements are within about 0.006.

Final model

The final model reached a test AUC of 0.978. Moreover, it classified 93.4% of test pairs correctly. It accepted 93.9% of genuine same-writer pairs and wrongly accepted 7.0% of different-writer pairs.

Table 9: Final Siamese model: test performance (session 5).

Measure	Value
AUC	0.977
Accuracy (cut-off chosen on validation)	0.929
True match rate (same writer correctly accepted)	0.955
False match rate (different writer wrongly accepted)	0.096
Distance cut-off	0.522

Table 10: Final model: confusion table for the test pairs.

	Different	Same
Different	2296	110
Same	245	2349

The table above shows that of the 2,488 same-writer test pairs, 2,335 were correctly accepted and 153 were wrongly rejected. Of the 2,512 different-writer pairs, 2,336 were correctly rejected and 176 were wrongly accepted. Overall the model made 329 errors in 5,000 decisions, split roughly evenly between the two types.